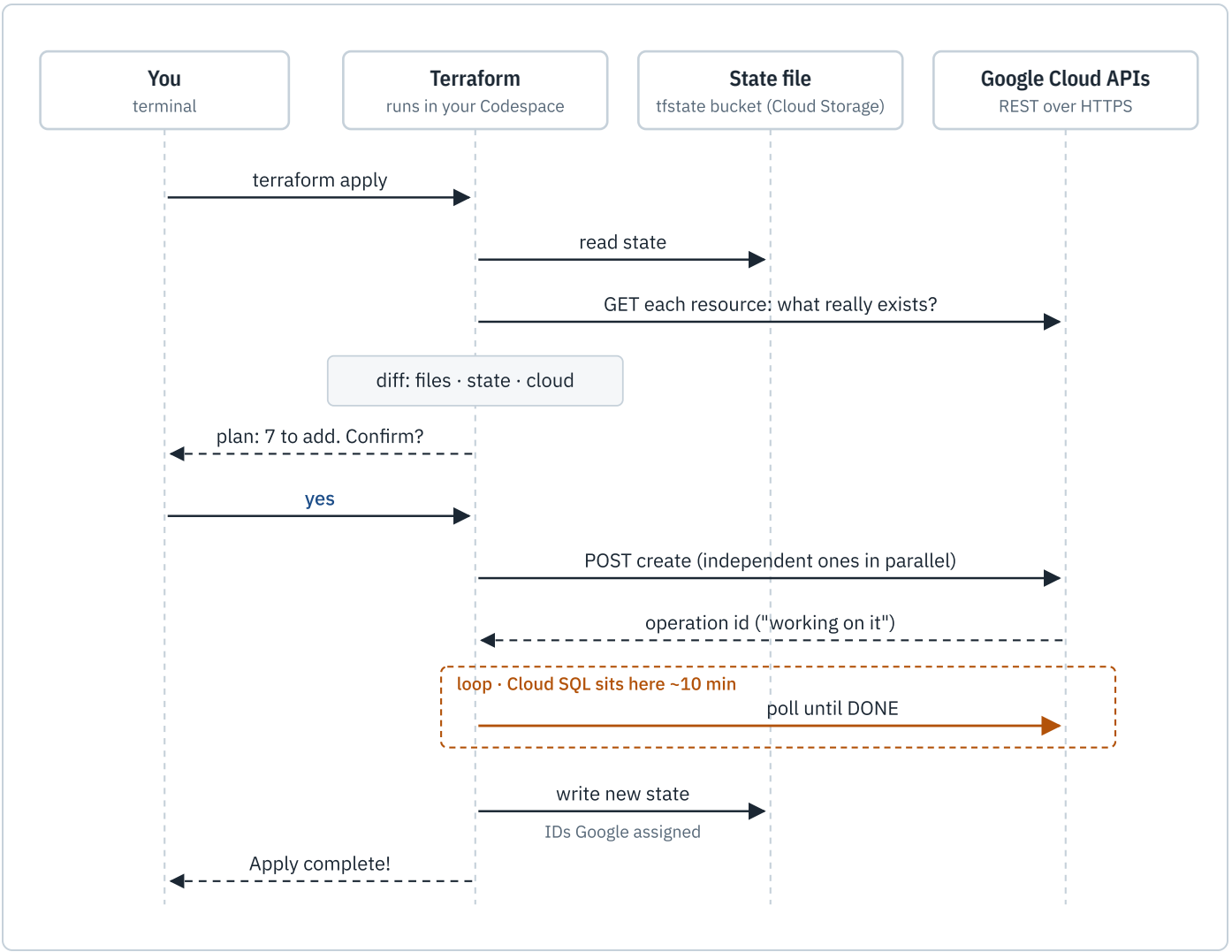
What we are building in Google Cloud, and how

You write `.tf` files describing what should exist. Terraform, running in your Codespace, turns that description into REST calls to Google, then records what it made. The result is a running copy of the app in Google's us-east1 data center, talking to a managed Postgres.

DIAGRAM 1

One `terraform apply`, call by call

This is exactly what happened when you applied the foundation, and what is happening now for the database. Nothing here touches your Codespace container or the app image. Terraform is just a client calling Google's APIs with your sign-in.



Solid arrows are requests, dashed arrows are replies. `terraform plan` is the first five steps only; it changes nothing. `docker push` follows the same pattern with a different client: REST calls to Artifact Registry, signed in as you.

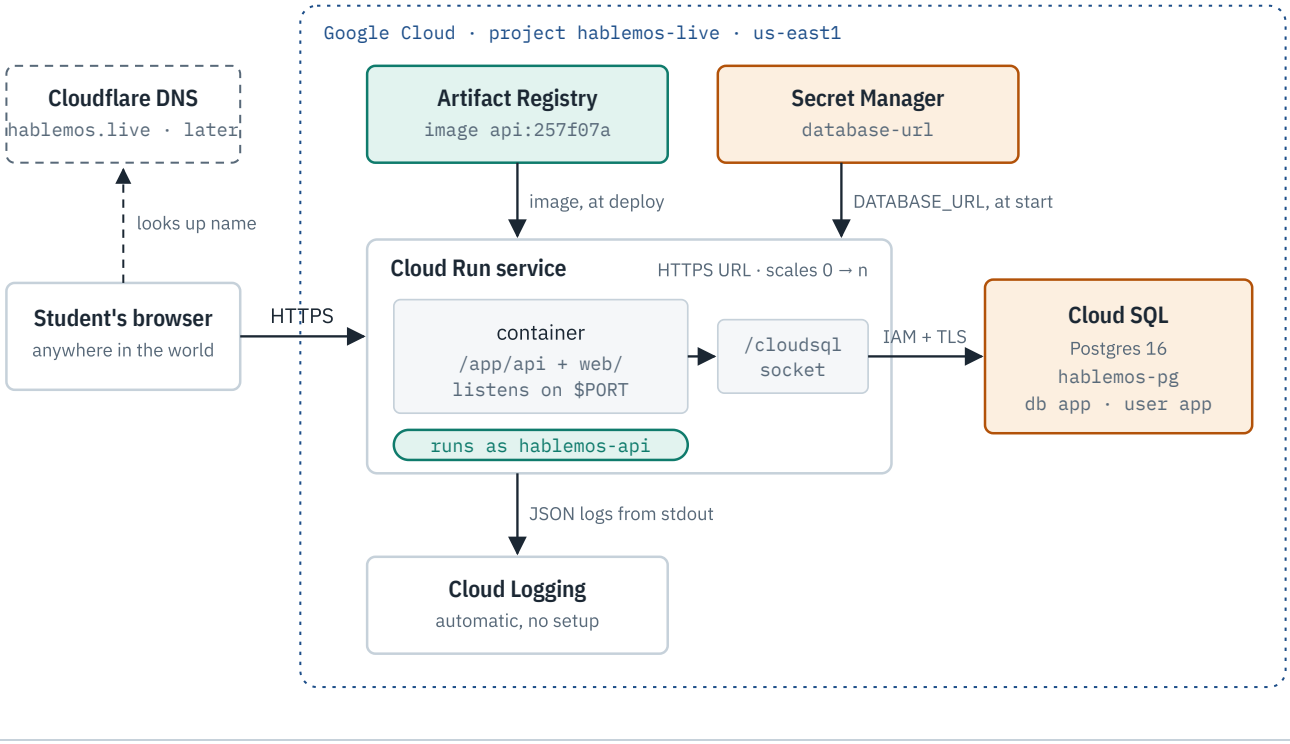
Is an apply atomic?

No, and this is the biggest difference from a database transaction. There is no commit or rollback. If an apply fails halfway, the resources that finished stay created and get recorded in state. You fix the cause and run `apply` again, and Terraform carries on from where reality is. Think of it as idempotent replay against a target state rather than a unit of work.

DIAGRAM 2

The running system, after step 4

Once deployed, Terraform is out of the picture. Google runs everything below, and your Codespace can be switched off.



exists now · being created by your current apply · step 4 (Cloud Run brings logging with it)

Terraform part 2

Until the domain is set up, students reach the app at a Google-made `…run.app` address. The connector socket is why the database needs no allowed networks: the only way in is a caller whose service account holds `cloudsql.client`.

Following one request

1. A student opens the site. Their browser sends `GET /sessions` over HTTPS to the Cloud Run URL. Google terminates TLS, so the Go server still speaks plain HTTP on `$PORT`, just like in the Codespace.
2. If no copy of the container is running (nobody has visited for a while), Cloud Run starts one. That first request waits a second or two. This is the "cold start", and it is the price of paying nothing while idle.
3. At startup the container already has `DATABASE_URL` in its environment. Cloud Run fetched it from Secret Manager, which it may do because the service runs as `hablemos-api`.
4. pgx opens the Unix socket at `/cloudsql/hablemos-live:us-east1:hablemos-pg`. The connector behind it checks the service account's IAM role, opens a TLS connection to Cloud SQL, and passes the Postgres protocol through.
5. The query runs, JSON goes back to the browser, and the request log line (with its request ID) lands in Cloud Logging.

INVENTORY

Every piece, and where it comes from

PIECE	WHAT IT IS	CLOSEST THING YOU KNOW	DEFINED IN	STATUS
State file	Terraform's record of what it created, with Google's IDs	A system catalog that the next run compares against	`tfstate bucket`	`exists`
Service APIs	Switches that let the project use a product	Licensing a product option on the system	`main.tf`	`on`
Artifact Registry	Private Docker image store, repository `app`	The library you deploy program objects from	`main.tf`	`image pushe`
Service account	`hablemos-api`, the identity the app runs as	The user profile a job runs under, with only its own authorities	`main.tf`	`exists`

Cloud SQL	A Google-managed VM running Postgres 16, with daily backups	The database server, with Google applying fixes and taking saves	`database.tf`	creating
Secret	`database-url`: the full connection string, password included	A credential store only one profile may read	`database.tf`	creating
IAM grants	`cloudsql.client` and `secretAccessor` for the service account	Object authorities granted to that profile	`database.tf`	creating
Cloud Run service	Runs the container on demand behind an HTTPS URL	A subsystem that starts jobs when work arrives and ends them when idle	`run.tf`	step 4

WHERE WE ARE

Next up

1. When the apply finishes, run the migrations against the new database once, from the Codespace, through the Cloud SQL Auth Proxy.
2. Add `run.tf`: the Cloud Run service using image `api:257f07a`, running as `hablemos-api`, with the secret as `DATABASE_URL`. Apply, then curl the `run.app` URL.
3. Terraform part 2: DNS records at Cloudflare and the domain mapping, so `https://hablemos.live` works.